Ingestion Engine Status Update

Reporting date: September 28, 2026
Branch reviewed: `feature/ingestion-engine-batch-lifecycle`
Remote reference: `origin/feature/ingestion-engine-batch-lifecycle`
Review scope: Code and documentation currently synchronized in the repository

Executive summary

The Ingestion Engine is currently a **configuration prototype with a partial backend foundation**, not a production ingestion engine. The frontend can guide an administrator through source-file inspection, draft field configuration, preview, versioned template management, identity settings, ingestion ordering, behavior metadata and Template Object relationships.

Those settings are persisted as local configuration evidence. They are not yet consumed by the production watcher, `IngestionOrchestrator`, source adapters, matching services or canonical Employee workflows. Saving or approving a template does not activate a watcher, import a file, create an object, resolve an identity or publish an authoritative change.

Current implementation status

Area	Status	Assessment
Source configuration page	Implemented	`/admin/ingestion-sources` supports the draft workflow and normal form fallback.
CSV/XLSX sample inspection	Implemented for bounded samples	File structure, worksheets, header candidates, composed headers and sample rows can be reviewed in memory.
Draft schema configuration	Implemented	Columns can be included or excluded; field keys, inferred types and safe cleanup suggestions can be reviewed or changed.
Draft validation and preview	Implemented	Structural diagnostics and normalized preview are available without creating production ingestion records.
Versioned report templates	Implemented	Draft and approved snapshots are stored with attribution and content hashes. Approved versions are immutable.
Identity governance settings	Implemented as configuration	Identity role, selected identifiers, creation permission and matching-field limits are persisted, but no runtime resolver consumes them.

Template ingestion order	Implemented as configuration	Order is stored separately from immutable template versions; it is not used to schedule or dispatch imports.

Area	Status	Assessment
Template behavior settings	Implemented as configuration	SharePoint subfolder, timezone, calendar/date fields and future skip metadata can be saved; no watcher or conversion logic uses them.
Template Object configuration	Implemented as configuration	Sources, properties, identifiers and cross-report match groups can be saved as versioned configuration. No object engine executes them.
Delete/restore lifecycle	Implemented as evidence retention	Removal and restoration events are retained; the operation does not delete historical template versions.
Generic batch lifecycle	Partial	`IngestionBatchService` exists and is covered by focused tests, but source-by-source production migration is incomplete.
Runtime SourceDefinition/SourceRegistry	Not implemented	No persisted runtime registry is connected to the watcher or orchestrator.
Runtime activation/publication	Not implemented	There is intentionally no activation, publish, enable-watcher or run-ingestion action in this phase.

Frontend review

The current frontend is more than a visual mockup, but it remains a governed configuration surface:

1. The source configuration screen reads bounded CSV/XLSX samples, supports worksheet selection, detects header candidates and composes multiple header rows into distinct source field names.
2. The draft mapping experience allows source-column inclusion/exclusion, editable field keys, inferred or overridden types and an allow-listed cleanup suggestion. Identifier values remain strings so leading zeroes are preserved.
3. The preview path returns diagnostics and normalized sample rows in memory. It explicitly reports that no production ingestion objects were created.
4. The template library provides cards and HTMX modals for identity settings, source ordering, behavior metadata, deletion/restoration and continuation of draft or approved versions.
5. The Template Object modal can select report sources, derive properties from included fields, select configured identifiers and create one-to-many cross-report match groups. The browser-side JavaScript recalculates the available choices and relationship preview after HTMX swaps.

The UI itself communicates the boundary through labels such as **Draft only**, **No watcher · No activation** and **Template design only**. This is consistent with the current Phase 1 read-only operating model.

Backend review

The implementation has the following supporting layers:

- `DraftConfigurationService` handles bounded sample inspection, header composition, type suggestions, draft validation and preview generation.
- `TemplateRepository` persists versioned source-template snapshots, approval state, identity governance, behavior metadata and ingestion order.
- `ObjectTemplateRepository` persists the named Template Object definition and retains source-field lineage in the existing employee-template history shape.
- `ingestion_configuration_routes.py` owns the administrative HTTP surface and keeps route handlers focused on validation, service calls and rendering.
- Migrations `0023` through `0027` add versioned definitions, order metadata, employee-template snapshots and auditable deletion/restoration events.
- `IngestionBatchService` provides a generic persistence boundary for batch metadata, records, source facts and counters. It is not yet the common path for every source.

The existing runtime remains separate. `IngestionOrchestrator` still dispatches by hard-coded source name to source-specific ingestion services. Roster Lock already delegates batch persistence to the generic batch service, while Master Roster, AccessNow and RSR still retain source-specific persistence paths. The legacy paths are active compatibility surfaces and have not been removed.

What is not connected yet

The following gaps prevent this branch from being described as a complete Ingestion Engine:

- Saved templates are not transformed into a runtime `SourceDefinition`.
- There is no runtime `SourceRegistry` that selects a definition for a source file or watcher event.
- The watcher does not use the configured SharePoint subfolder, order, timezone, date fields or skip metadata.
- The orchestrator does not dispatch through the template library.
- The Template Object is not an execution engine: it does not create objects, resolve identities, define merge precedence or map source fields into canonical Employee attributes.
- Production matching and reconciliation remain in existing source-specific services and Employee workflows.
- Batch policy is not pinned to an immutable definition snapshot at import time.
- AccessNow and RSR have not been migrated to the generic batch lifecycle.
- There is no production activation/publication workflow, by design.

Recommended next sequence

1. Freeze the current configuration contract with focused parity fixtures.
2. Define the runtime `SourceDefinition` adapter and `SourceRegistry` without changing the existing watcher or source behavior.
3. Pin a selected definition version into each future batch before execution.
4. Use Roster Lock as the first controlled runtime pilot because its current path is comparatively linear and already uses `IngestionBatchService`.
5. Compare old and pilot paths for counts, lineage, source facts, statuses, deduplication and retry behavior.
6. Migrate additional sources incrementally only after parity is demonstrated.

Any later activation or publication workflow requires an explicit scope and authorization decision. It must remain separate from the current read-only configuration review.

Conclusion

The branch has delivered a credible administrator-facing configuration layer and the first reusable batchlifecycle building block. The main architectural boundary is still intact: the new frontend and its persisted settings are not the production ingestion path. The next meaningful milestone is runtime definition resolution and a measured Roster Lock pilot, not additional configuration controls alone.